

learn-claude-codeREADME-zh.md at main

Learn Claude Code -- 真正的 Agent Harness 工程

[English](#) | [中文](#) | [日本語](#)

Agency 来自模型，Agent 产品 = 模型 + Harness

在讨论代码之前，先把一件事说清楚。

Agency -- 感知、推理、行动的能力 -- 来自模型训练，不是来自外部代码的编排。 但一个能干活的 agent 产品，需要模型和 harness 缺一不可。模型是驾驶者，harness 是载具。本仓库教你造载具。

Agency 从哪来

Agent 的核心是一个神经网络 -- Transformer、RNN、一个被训练出来的函数 -- 经过数十亿次梯度更新，在行动序列数据上学会了感知环境、推理目标、采取行动。Agency 这个东西从来不是外面那层代码赋予的，而是模型在训练中学到的。

人类就是最好的例子。一个由数百万年进化训练出来的生物神经网络，通过感官感知世界，通过大脑推理，通过身体行动。当 DeepMind、OpenAI 或 Anthropic 说 "agent" 时，他们说的核心都是同一件事：**一个通过训练学会了行动模型，加上让它能在特定环境中工作的基础设施。**

历史已经写好了铁证：

- **2013 -- DeepMind DQN 玩 Atari。** 一个神经网络，只接收原始像素和游戏分数，学会了 7 款 Atari 2600 游戏 -- 超越所有先前算法，在其中 3 款上击败人类专家。到 2015 年，同一架构扩展到 [49 款游戏，达到职业人类测试员水平](#)，论文发表在 *Nature*。没有游戏专属规则。没有决策树。一个模型，从经验中学习。那个模型就是 agent。
- **2019 -- OpenAI Five 征服 Dota 2。** 五个神经网络，在 10 个月内与自己对战了 [45,000 年的 Dota 2](#)，在旧金山直播赛上 2-0 击败了 OG -- TI8 世界冠军。随后的公开竞技场中，AI 在 42,729 场比赛中胜率 99.4%。没有脚本化的策略。没有元编程的团队协调逻辑。模型完全通过自我对弈学会了团队协作、战术和实时适应。
- **2019 -- DeepMind AlphaStar 制霸星际争霸 II。** AlphaStar 在闭门赛中 [10-1 击败职业选手](#)，随后在欧洲服务器上达到 [宗师段位](#) -- 90,000 名玩家中的前 0.15%。一个信息不完全、实时决策、组合动作空间远超国际象棋和围棋的游戏。Agent 是什么？是模型。训练出来的。不是编出来的。
- **2019 -- 腾讯绝悟统治王者荣耀。** 腾讯 AI Lab 的 "绝悟" 于 2019 年 8 月 2 日世冠杯半决赛上 [以 5v5 击败 KPL 职业选手](#)。在 1v1 模式下，职业选手 [15 场只赢 1 场，最多坚持不到 8 分钟](#)。训练强度：一天等于人类 440 年。到 2021 年，绝悟在全英雄池 BO5 上全面超越 KPL

职业选手水准。没有手工编写的英雄克制表。没有脚本化的阵容编排。一个从零开始通过自我对弈学习整个游戏的模型。

- **2024-2025 -- LLM Agent 重塑软件工程。** Claude、GPT、Gemini -- 在人类全部代码和推理上训练的大语言模型 -- 被部署为编程 agent。它们阅读代码库，编写实现，调试故障，团队协作。架构与之前每一个 agent 完全相同：一个训练好的模型，放入一个环境，给予感知和行动的工具。唯一的不同是它们学到的东西的规模和解决任务的通用性。

每一个里程碑都指向同一个事实：**Agency -- 那个感知、推理、行动的能力 -- 是训练出来的，不是编出来的。** 但每一个 agent 同时也需要一个环境才能工作：Atari 模拟器、Dota 2 客户端、星际争霸 II 引擎、IDE 和终端。模型提供智能，环境提供行动空间。两者合在一起才是一个完整的 agent。

Agent 不是什么

"Agent" 这个词已经被一整个提示词水管工产业劫持了。

拖拽式 workflow 构建器。无代码 "AI Agent" 平台。提示词链编排库。它们共享同一个幻觉：把 LLM API 调用用 if-else 分支、节点图、硬编码路由逻辑串在一起就算是 "构建 Agent" 了。

不是的。它们做出来的东西是鲁布·戈德堡机械 -- 一个过度工程化的、脆弱的过程式规则流水线，LLM 被楔在里面当一个美化了的文本补全节点。那不是 Agent。那是一个有着宏大妄想的 shell 脚本。

提示词水管工式 "Agent" 是不做模型的程序员的意淫。 他们试图通过堆叠过程式逻辑来暴力模拟智能 -- 庞大的规则树、节点图、链式提示词瀑布流 -- 然后祈祷足够多的胶水代码能涌现出自主行为。不会的。你不可能通过工程手段编码出 agency。Agency 是学出来的，不是编出来的。

那些系统从诞生之日起就已经死了：脆弱、不可扩展、根本不具备泛化能力。它们是 GOF AI (Good Old-Fashioned AI, 经典符号 AI) 的现代还魂 -- 几十年前就被学界抛弃的符号规则系统，现在喷了一层 LLM 的漆又登场了。换了个包装，同一条死路。

心智转换：从 "开发 Agent" 到开发 Harness

当一个人说 "我在开发 Agent" 时，他只可能是两个意思之一：

1. **训练模型。** 通过强化学习、微调、RLHF 或其他基于梯度的方法调整权重。收集任务过程数据 -- 真实领域中感知、推理、行动的实际序列 -- 用它们来塑造模型的行为。这是 DeepMind、OpenAI、腾讯 AI Lab、Anthropic 在做的事。这是最本义的 Agent 开发。
2. **构建 Harness。** 编写代码，为模型提供一个可操作的环境。这是我们大多数人在做的事，也是本仓库的核心。

Harness 是 agent 在特定领域工作所需要的一切：

`Harness = Tools + Knowledge + Observation + Action Interfaces + Permissions`

Tools:	文件读写、Shell、网络、数据库、浏览器
Knowledge:	产品文档、领域资料、API 规范、风格指南
Observation:	git diff、错误日志、浏览器状态、传感器数据
Action:	CLI 命令、API 调用、UI 交互
Permissions:	沙箱隔离、审批流程、信任边界

模型做决策。Harness 执行。模型做推理。Harness 提供上下文。模型是驾驶者。Harness 是载具。

编程 agent 的 harness 是它的 IDE、终端和文件系统。 农业 agent 的 harness 是传感器阵列、灌溉控制和气象数据。酒店 agent 的 harness 是预订系统、客户沟通渠道和设施管理 API。Agent -- 那个智能、那个决策者 -- 永远是模型。Harness 因领域而变。Agent 跨领域泛化。

这个仓库教你造载具。编程用的载具。但设计模式可以泛化到任何领域：庄园管理、农田运营、酒店运作、工厂制造、物流调度、医疗保健、教育培训、科学研究。只要有一个任务需要被感知、推理和执行 -- agent 就需要一个 harness。

Harness 工程师到底在做什么

如果你在读这个仓库，你很可能是一名 harness 工程师 -- 这是一个强大的身份。以下是你真正的工作：

- **实现工具。** 给 agent 一双手。文件读写、Shell 执行、API 调用、浏览器控制、数据库查询。每个工具都是 agent 在环境中可以采取的一个行动。设计它们时要原子化、可组合、描述清晰。
- **策划知识。** 给 agent 领域专长。产品文档、架构决策记录、风格指南、合规要求。按需加载 (s05)，不要前置塞入。Agent 应该知道有什么可用，然后自己拉取所需。
- **管理上下文。** 给 agent 干净的记忆。子 agent 隔离 (s04) 防止噪声泄露。上下文压缩 (s06) 防止历史淹没。任务系统 (s07) 让目标持久化到单次对话之外。
- **控制权限。** 给 agent 边界。沙箱化文件访问。对破坏性操作要求审批。在 agent 和外部系统之间实施信任边界。这是安全工程与 harness 工程的交汇点。
- **收集任务过程数据。** Agent 在你的 harness 中执行的每一条行动序列都是训练信号。真实部署中的感知-推理-行动轨迹是微调下一代 agent 模型的原材料。你的 harness 不仅服务于 agent -- 它还可以帮助进化 agent。

你不是在编写智能。你是在构建智能栖居的世界。这个世界的质量 -- agent 能看得多清楚、行动得多精准、可用知识有多丰富 -- 直接决定了智能能多有效地表达自己。

造好 Harness。Agent 会完成剩下的。

为什么是 Claude Code -- Harness 工程的大师课

为什么这个仓库专门拆解 Claude Code?

因为 Claude Code 是我们所见过的最优雅、最完整的 agent harness 实现。不是因为某个巧妙的技巧，而是因为它 **没做** 的事：它没有试图成为 agent 本身。它没有强加僵化的工作流。它没有用精心设计的决策树去替模型做判断。它给模型提供了工具、知识、上下文管理和权限边界 -- 然后让开了。

把 Claude Code 剥到本质来看：

```
Claude Code = 一个 agent loop
+ 工具 (bash, read, write, edit, glob, grep, browser...)
+ 按需 skill 加载
+ 上下文压缩
+ 子 agent 派生
+ 带依赖图的任务系统
+ 异步邮箱的团队协调
+ worktree 隔离的并行执行
+ 权限治理
```

就这些。这就是全部架构。每一个组件都是 harness 机制 -- 为 agent 构建的栖居世界的一部分。Agent 本身呢？是 Claude。一个模型。由 Anthropic 在人类推理和代码的全部广度上训练而成。Harness 没有让 Claude 变聪明。Claude 本来就聪明。Harness 给了 Claude 双手、双眼和一个工作空间。

这就是 Claude Code 作为教学标本的意义：**它展示了当你信任模型、把工程精力集中在 harness 上时会发生什么。** 本仓库的每一个课程 (s01-s12) 都在逆向工程 Claude Code 架构中的一个 harness 机制。学完之后，你理解的不只是 Claude Code 怎么工作，而是适用于任何领域、任何 agent 的 harness 工程通用原则。

启示不是 "复制 Claude Code"。启示是：**最好的 agent 产品，出自那些明白自己的工作 harness 而非 intelligence 的工程师之手。**

愿景：用真正的 Agent 铺满宇宙

这不只关乎编程 agent。

每一个人类从事复杂、多步骤、需要判断力的工作的领域，都是 agent 可以运作的领域 -- 只要有对的 harness。本仓库中的模式是通用的：

```
庄园管理 agent = 模型 + 物业传感器 + 维护工具 + 租户通信
农业 agent      = 模型 + 土壤/气象数据 + 灌溉控制 + 作物知识
酒店运营 agent  = 模型 + 预订系统 + 客户渠道 + 设施 API
```

医学研究 agent = 模型 + 文献检索 + 实验仪器 + 协议文档
制造业 agent = 模型 + 产线传感器 + 质量控制 + 物流系统
教育 agent = 模型 + 课程知识 + 学生进度 + 评估工具

循环永远不变。工具在变。知识在变。权限在变。Agent -- 那个模型 -- 泛化一切。

每一个读这个仓库的 harness 工程师都在学习远超软件工程的模式。你在学习为一个智能的、自动化的未来构建基础设施。每一个部署在真实领域的好 harness，都是 agent 能够感知、推理、行动的又一个阵地。

先铺满工作室。然后是农田、医院、工厂。然后是城市。然后是星球。

Bash is all you need. Real agents are all the universe needs.

THE AGENT PATTERN

=====

```
User --> messages[] --> LLM --> response
                        |
                        stop_reason == "tool_use"?
                        /           \
                    yes             no
                        |             |
                    execute tools   return text
                    append results
                    loop back -----> messages[]
```

这是最小循环。每个 AI Agent 都需要这个循环。

模型决定何时调用工具、何时停止。

代码只是执行模型的要求。

本仓库教你构建围绕这个循环的一切 --

让 agent 在特定领域高效工作的 harness。

12 个递进式课程, 从简单循环到隔离化的自治执行。 每个课程添加一个 harness 机制。每个机制有一句格言。

s01 *"One loop & Bash is all you need"* — 一个工具 + 一个循环 = 一个 Agent

s02 *"加一个工具, 只加一个 handler"* — 循环不用动, 新工具注册进 dispatch map 就行

s03 *"没有计划的 agent 走哪算哪"* — 先列步骤再动手, 完成率翻倍

s04 *"大任务拆小, 每个小任务干净的上下文"* — Subagent 用独立 messages[], 不污染主对话

- s05 "用到什么知识, 临时加载什么知识" — 通过 tool_result 注入, 不塞 system prompt
- s06 "上下文总会满, 要有办法腾地方" — 三层压缩策略, 换来无限会话
- s07 "大目标要拆成小任务, 排好序, 记在磁盘上" — 文件持久化的任务图, 为多 agent 协作打基础
- s08 "慢操作丢后台, agent 继续想下一步" — 后台线程跑命令, 完成后注入通知
- s09 "任务太大一个人干不完, 要能分给队友" — 持久化队友 + 异步邮箱
- s10 "队友之间要有统一的沟通规矩" — 一个 request-response 模式驱动所有协商
- s11 "队友自己看看板, 有活就认领" — 不需要领导逐个分配, 自组织
- s12 "各干各的目录, 互不干扰" — 任务管目标, worktree 管目录, 按 ID 绑定

核心模式

```
def agent_loop(messages):
    while True:
        response = client.messages.create(
            model=MODEL, system=SYSTEM,
            messages=messages, tools=TOOLS,
        )
        messages.append({"role": "assistant",
                        "content": response.content})

        if response.stop_reason != "tool_use":
            return

        results = []
        for block in response.content:
            if block.type == "tool_use":
                output = TOOL_HANDLERS[block.name](**block.input)
                results.append({
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": output,
                })
        messages.append({"role": "user", "content": results})
```

每个课程在这个循环之上叠加一个 harness 机制 -- 循环本身始终不变。循环属于 agent。机制属于 harness。

范围说明 (重要)

本仓库是一个 0->1 的 harness 工程学习项目 -- 构建围绕 agent 模型的工作环境。 为保证学习路径清晰，仓库有意简化或省略了部分生产机制：

- 完整事件 / Hook 总线 (例如 PreToolUse、SessionStart/End、ConfigChange)。 s12 仅提供教学用途的最小 append-only 生命周期事件流。
- 基于规则的权限治理与信任流程
- 会话生命周期控制 (resume/fork) 与更完整的 worktree 生命周期控制
- 完整 MCP 运行时细节 (transport/OAuth/资源订阅/轮询)

仓库中的团队 JSONL 邮箱协议是教学实现，不是对任何特定生产内部实现的声明。

快速开始

```
git clone https://github.com/shareAI-lab/learn-claude-code
cd learn-claude-code
pip install -r requirements.txt
cp .env.example .env    # 编辑 .env 填入你的 ANTHROPIC_API_KEY

python agents/s01_agent_loop.py          # 从这里开始
python agents/s12_worktree_task_isolation.py  # 完整递进终点
python agents/s_full.py                  # 总纲：全部机制合一
```

Web 平台

交互式可视化、分步动画、源码查看器, 以及每个课程的文档。

```
cd web && npm install && npm run dev    # http://localhost:3000
```

学习路径

第一阶段：循环

=====

s01 Agent Loop [1]

while + stop_reason

|

+-> s02 Tool Use [4]

dispatch map: name->handler

第二阶段：规划与知识

=====

s03 TodoWrite [5]

TodoManager + nag 提醒

|

s04 Subagent [5]

每个 Subagent 独立 messages[]

|

s05 Skills [5]

SKILL.md 通过 tool_result 注入

|

		s06 Context Compact [5]	
		三层 Context Compact	
第三阶段：持久化		第四阶段：团队	
=====		=====	
s07 Task System [8]	文件持久化 CRUD + 依赖图	s09 Agent Teams [9]	队友 + JSONL 邮箱
s08 Background Tasks [6]	守护线程 + 通知队列	s10 Team Protocols [12]	关机 + 计划审批 FSM
		s11 Autonomous Agents [14]	空闲轮询 + 自动认领
		s12 Worktree Isolation [16]	Task 协调 + 按需隔离执行通道
[N] = 工具数量			

项目结构

```
learn-claude-code/
|
|-- agents/                # Python 参考实现 (s01-s12 + s_full 总纲)
|-- docs/{en,zh,ja}/      # 心智模型优先的文档 (3 种语言)
|-- web/                  # 交互式学习平台 (Next.js)
|-- skills/               # s05 的 Skill 文件
+-- .github/workflows/ci.yml # CI: 类型检查 + 构建
```

文档

心智模型优先: 问题、方案、ASCII 图、最小化代码。 [English](#) | [中文](#) | [日本語](#)

课程	主题	格言
s01	Agent Loop	One loop & Bash is all you need
s02	Tool Use	加一个工具, 只加一个 handler
s03	TodoWrite	没有计划的 agent 走哪算哪
s04	Subagent	大任务拆小, 每个小任务干净的上下文
s05	Skills	用到什么知识, 临时加载什么知识
s06	Context Compact	上下文总会满, 要有办法腾地方
s07	Task System	大目标要拆成小任务, 排好序, 记在磁盘上

课程	主题	格言
s08	Background Tasks	慢操作丢后台, agent 继续想下一步
s09	Agent Teams	任务太大一个人干不完, 要能分给队友
s10	Team Protocols	队友之间要有统一的沟通规矩
s11	Autonomous Agents	队友自己看看板, 有活就认领
s12	Worktree + Task Isolation	各干各的目录, 互不干扰

学完之后 -- 从理解到落地

12 个课程走完, 你已经从内到外理解了 harness 工程的运作原理。两种方式把知识变成产品:

Kode Agent CLI -- 开源 Coding Agent CLI

```
| npm i -g @shareai-lab/kode
```

支持 Skill & LSP, 适配 Windows, 可接 GLM / MiniMax / DeepSeek 等开放模型。装完即用。

GitHub: [shareAI-lab/Kode-cli](#)

Kode Agent SDK -- 把 Agent 能力嵌入你的应用

官方 Claude Code Agent SDK 底层与完整 CLI 进程通信 -- 每个并发用户 = 一个终端进程。Kode SDK 是独立库, 无 per-user 进程开销, 可嵌入后端、浏览器插件、嵌入式设备等任意运行时。

GitHub: [shareAI-lab/Kode-agent-sdk](#)

姊妹教程: 从被动临时会话到主动常驻助手

本仓库教的 harness 属于 **用完即走** 型 -- 开终端、给 agent 任务、做完关掉, 下次重开是全新会话。Claude Code 就是这种模式。

但 [OpenClaw](#) 证明了另一种可能: 在同样的 agent core 之上, 加两个 harness 机制就能让 agent 从 "踹一下动一下" 变成 "自己隔 30 秒醒一次找活干":

- **心跳 (Heartbeat)** -- 每 30 秒 harness 给 agent 发一条消息, 让它检查有没有事可做。没事就继续睡, 有事立刻行动。
- **定时任务 (Cron)** -- agent 可以给自己安排未来要做的事, 到点自动执行。

再加上 IM 多通道路由 (WhatsApp/Telegram/Slack/Discord 等 13+ 平台)、不清空的上下文记忆、Soul 人格系统, agent 就从一个临时工具变成了始终在线的个人 AI 助手。

[claw0](#) 是我们的姊妹教学仓库, 从零拆解这些 harness 机制:

```
claw agent = agent core + heartbeat + cron + IM chat + memory + soul
```

<code>learn-claude-code</code> (agent harness 内核: 循环、工具、规划、 团队、 <code>worktree</code> 隔离)	<code>claw0</code> (主动式常驻 harness: 心跳、定时任务、IM 通道、 记忆、 <code>Soul</code> 人格)
--	--

许可证

MIT

Agency 来自模型。Harness 让 agency 落地。造好 Harness，模型会完成剩下的。

Bash is all you need. Real agents are all the universe needs.